

React Router

1. What Is React Router,

Normally, a website is made of many separate HTML pages. When you click a link, the browser throws away the current page and loads a brand new one from the server. That's why you sometimes see a quick white flash — the whole page reloads.

A React app is different — it's a "single-page application". There's really only ONE html page. Everything you see is swapped in and out by JavaScript, without ever refreshing the browser. That makes apps feel fast and smooth, like a native app.

But here's the problem: if there's only one page, how does the app know to show your Profile when you go to "/profile" and your Inbox when you go to "/inbox"? That's exactly what React Router solves. It watches the web address (URL) and decides which piece of the app to display, without ever asking the server for a new page.

Think of it like this: React Router is like a receptionist standing at the entrance of a building. They don't build new rooms — they just look at who walked in (the URL) and point them to the right room (the right component).

2. Installing It

Before you can use it, you add it to your project with one command in the terminal:

```
npm install react-router-dom
```

Once installed, you'll import the pieces you need from "react-router-dom" whenever you use them.

3. Step One: Wrap Your App in a Router

For any of this to work, your entire app needs to sit inside a special wrapper component called BrowserRouter. This wrapper is what gives your app "URL awareness" — without it, none of the routing features will work.

```
// index.js / main.jsx
import { BrowserRouter } from "react-router-dom";
import App from "./App";

ReactDOM.createRoot(document.getElementById("root")).render(
  <BrowserRouter>
    <App />
  </BrowserRouter>
);
```

Think of it like this: This is like plugging in the building's directory board before the receptionist can use it — the app can't route people around until this wrapper is in place.

Good to know: There are a few other wrapper options (HashRouter, MemoryRouter) for special cases, but BrowserRouter is the one you'll use for almost every normal website.

4. Step Two: Decide What Shows for Each URL

Inside your app, you list out each URL "path" and which component should appear for it. You do this with two components working together: Routes (the overall list) and Route (one single rule in that list).

```
import { Routes, Route } from "react-router-dom";

function App() {
```

```

return (
  <Routes>
    <Route path="/" element={<Home />} />
    <Route path="/about" element={<About />} />
    <Route path="/contact" element={<Contact />} />
    <Route path="*" element={<NotFound />} />
  </Routes>
);
}

```

Read this like a set of instructions: "If the URL is exactly '/', show Home. If it's '/about', show About. If nothing else matches, show NotFound."

Good to know: The `path="*" element={<NotFound />} />` line is a catch-all — a safety net for any URL that doesn't match anything else. It's how you build a "Page Not Found" (404) screen. Always put it last.

5. Letting People Click Around: Link and NavLink

You might think you should use a normal HTML `<a>` tag to make links. Don't — a regular link makes the browser do a full page reload, which defeats the whole point. Instead, React Router gives you its own version called `Link`, which looks and works like a link but quietly swaps the view instead of reloading the page.

```

import { Link } from "react-router-dom";

<nav>
  <Link to="/">Home</Link>
  <Link to="/about">About</Link>
</nav>

```

`NavLink` is the same idea, but it automatically knows when it's the "current" page and can style itself differently — handy for highlighting the active tab in a navigation menu.

```

import { NavLink } from "react-router-dom";

<NavLink
  to="/about"
  className={({ isActive }) => (isActive ? "active-link" : "")}
>
  About
</NavLink>

```

6. Sending People to a New Page from Code

Sometimes you want to redirect someone without them clicking a link — for example, after they successfully log in, or submit a form. For this, use the `useNavigate` hook, which gives you a function you can call whenever you want.

```

import { useNavigate } from "react-router-dom";

function LoginForm() {
  const navigate = useNavigate();

  function handleSubmit() {
    // ...check the login details
    navigate("/dashboard"); // send them here after logging in
  }

  return <button onClick={handleSubmit}>Log in</button>;
}

```

```
}
```

Think of it like this: If `Link` is a doorway a visitor walks through themselves, `useNavigate` is you personally walking them there.

7. Pages with Changing Parts: Dynamic Routes

What if you have thousands of user profiles, and you don't want to write a separate route for every single one? You use a placeholder in the path — written with a colon — that can match anything.

```
<Route path="/users/:userId" element={<UserProfile />} />
```

Now any URL like `/users/42` or `/users/emma` will match this one route. Inside the component, you can read out whatever value was actually typed using the `useParams` hook:

```
import { useParams } from "react-router-dom";

function UserProfile() {
  const { userId } = useParams();
  return <h2>Viewing user: {userId}</h2>;
}
```

Think of it like this: It's like a mail slot labeled `'userId'` — whatever name gets dropped in, the component can read it off the envelope.

8. Reading Extra Info from the URL

Two more hooks are useful when you need more detail about where the user currently is:

`useLocation`

Gives you an object describing the current URL — useful if you just want to know exactly where you are right now.

```
import { useLocation } from "react-router-dom";

const location = useLocation();
console.log(location.pathname); // e.g. "/about"
```

`useSearchParams`

Handles the part of a URL after a question mark, like `?sort=price` — often used for filters, search, or sorting options.

```
import { useSearchParams } from "react-router-dom";

const [searchParams, setSearchParams] = useSearchParams();
const sort = searchParams.get("sort");

setSearchParams({ sort: "price" });
```

9. Pages Inside Pages: Nested Routes

Real apps often have a layout that stays the same (like a sidebar or header) while only the middle part of the screen changes. React Router handles this with nested routes and a placeholder called `Outlet`, which marks the spot where the "inner" page should appear.

```

<Routes>
  <Route path="/dashboard" element={<DashboardLayout />}>
    <Route index element={<DashboardHome />} />
    <Route path="settings" element={<Settings />} />
    <Route path="orders" element={<Orders />} />
  </Route>
</Routes>

import { Outlet } from "react-router-dom";

function DashboardLayout() {
  return (
    <div>
      <Sidebar />
      <Outlet /> { /* the matching child page appears here */ }
    </div>
  );
}

```

Think of it like this: Think of a picture frame (the layout) that never changes, while the photo inside it (Outlet) gets swapped depending on the page.

Good to know: The "index" route with no path is what shows by default — e.g. simply visiting /dashboard with nothing after it.

10. Blocking Pages Unless You're Logged In

A very common need is to stop someone from viewing a page (like a dashboard) unless they're logged in. You do this by checking a condition, and using the Navigate component to bounce them somewhere else if it fails.

```

import { Navigate, Outlet } from "react-router-dom";

function ProtectedRoute({ isAuthenticated }) {
  if (!isAuthenticated) {
    return <Navigate to="/login" replace />;
  }
  return <Outlet />;
}

<Route element={<ProtectedRoute isAuthenticated={user != null} />}>
  <Route path="/dashboard" element={<Dashboard />} />
</Route>

```

If the person isn't logged in, they get quietly redirected to /login instead of seeing the protected page. The replace option means clicking "back" won't take them into the page they were just blocked from.

11. Cheat Sheet

Name	What it is	Plain-English job
BrowserRouter	Component	Wraps the whole app so routing works
Routes / Route	Component	The list of rules: which URL shows which component
Link / NavLink	Component	Clickable links that don't reload the page

Name	What it is	Plain-English job
Outlet	Component	The empty slot where a child page appears
Navigate	Component	Automatically redirects the user
useNavigate	Hook	Redirects the user from inside your code
useParams	Hook	Reads values from the URL, like an ID
useLocation	Hook	Tells you exactly where the user currently is
useSearchParams	Hook	Reads/writes the ?key=value part of a URL

12. The Big Picture, in One Paragraph

Wrap the app in `BrowserRouter` so it understands URLs. List your pages with `Routes` and `Route`. Let people click between them with `Link` or `NavLink`, or send them there yourself with `useNavigate`. If a page needs to know an ID from the URL, read it with `useParams`. If pages share a layout, nest the routes and use `Outlet`. And if a page should be private, check a condition and redirect with `Navigate`. That's really the whole toolkit — everything else in React Router builds on these basics.